

RAG-LLM Chatbot

System Design & Feature Specification

FastAPI · PostgreSQL · pgvector · Hugging Face LLM

1. Overview

This document specifies the complete feature set for the RAG-LLM chatbot platform and the workflow behind each feature. It is organized into six functional areas: authentication and account management, conversation management, messaging, file and document handling, retrieval-augmented generation, and platform operations. Every feature is mapped to the underlying PostgreSQL schema, so this document also serves as the bridge between the database design and the FastAPI implementation.

The system's core loop is: a user authenticates, starts or resumes a conversation, optionally attaches documents for the assistant to reference, exchanges messages with the LLM, and the platform tracks provenance (which model, which retrieved chunks, how many tokens) for every response.

2. Authentication & Account Management

2.1 Why JWT Instead of Server-Side Sessions

This is a deliberate architectural decision worth explaining on its own, since it shapes the entire auth module and the `user_sessions` table.

A traditional server-side session works by generating a random session ID at login, storing it in the database (or an in-memory store like Redis), and handing that ID to the client as a cookie. Every subsequent request requires the server to look up that ID against the store to confirm who is authenticated. This is simple to reason about, and revocation is trivial — deleting the row instantly invalidates the session.

A pure JWT (JSON Web Token) approach works differently: at login, the server issues a signed token containing the user's identity and claims directly inside the token itself (user ID, expiry, etc.). Because the token is cryptographically signed, the server can verify it on every request without touching the database at all — verification is just signature checking, which is fast and requires no round trip to Postgres. For a chatbot backend serving many small requests per conversation (send message, load history, poll status, stream tokens), avoiding a database lookup on every single authenticated request is a meaningful performance win. It also decouples authentication from the database, which matters if the system is ever split into multiple services, since any service can verify a JWT independently without querying a shared session store.

The cost of pure JWT is that it has no built-in revocation. Once issued, a token remains valid until it expires, with no way for the server to force it invalid early — there is no row to delete. This means logout does not really work in a pure-JWT system, and a leaked long-lived token stays dangerous until it naturally expires.

The system resolves this with a hybrid model, which is why the schema has a `user_sessions` table at all. Two tokens are issued at login: a short-lived JWT access token (minutes to roughly an hour), used on the majority of requests exactly like pure JWT, with no database lookup; and a long-lived refresh token, which is a random opaque string stored server-side in `user_sessions`. The refresh token is checked against the database only when the access token expires — occasionally, not on every request — at which point the server confirms the session is not revoked and not past its expiry before issuing a new access token.

This hybrid gives the system JWT's main advantage, avoiding a database hit on nearly every request, while still supporting real logout, "log out of all devices," and session listing, since `is_revoked` can be flipped on the stored refresh token at any time. The `is_revoked`, `device`, and `ip_address` columns exist specifically to support this: they let a user see and manage active sessions, and let the server kill one specific session without logging the user out everywhere. A pure server-side session model would be simpler to reason about and would be a reasonable choice for a smaller, single-service application, but the hybrid was chosen here because it scales better under load while preserving the control that pure JWT gives up.

2.2 Features

2.2 Sign Up

Registers a new user account with a hashed password.

Workflow

1. Client submits username, email, and password.
2. Server checks the case-insensitive unique indexes on users.email and users.username for collisions.
3. Password is hashed with bcrypt (never stored in plaintext) and inserted into the users table.
4. Server issues an access token and refresh token immediately, or requires email verification first, depending on product requirements.

Tables involved: *users, user_sessions*

2.3 Login

Authenticates an existing user and starts a session.

Workflow

5. Client submits email and password.
6. Server looks up the user by lowercased email and verifies the password against password_hash.
7. On success, server issues a short-lived JWT access token and a long-lived refresh token.
8. Refresh token is stored in user_sessions along with device and ip_address for session management.

Tables involved: *users, user_sessions*

2.4 Token Refresh

Exchanges a valid refresh token for a new access token without re-entering credentials.

Workflow

9. Client sends its refresh token once the access token has expired.
10. Server checks user_sessions: the token exists, is_revoked is false, and expires_at has not passed.
11. Server issues a new access token (and optionally rotates the refresh token for additional security).

Tables involved: *user_sessions*

2.5 Logout

Ends a session, either on one device or everywhere.

Workflow

12. Client sends its current refresh token to the logout endpoint.
13. Server sets is_revoked = true on that specific user_sessions row.
14. "Log out everywhere" variant sets is_revoked = true on all sessions belonging to that user_id.

Tables involved: *user_sessions*

2.6 Password Reset / Account Recovery

Lets a user regain access without knowing their current password.

Workflow

15. User requests a reset via their email address.
16. Server generates a short-lived, single-use reset token (stored hashed, similar in spirit to user_sessions) and emails it to the user.
17. User submits a new password along with the token.

18. Server verifies the token, updates password_hash, invalidates the reset token, and revokes all existing user_sessions rows for that user, forcing re-login everywhere in case the account was compromised.

Tables involved: *users, password_reset_tokens (new table), user_sessions*

3. Conversations

3.1 Create Conversation

Starts a new chat thread for a user.

Workflow

19. User initiates a new chat, optionally choosing a model.
20. Server inserts a conversations row with a default title, the chosen model_name, and an empty generation_config.
21. Title remains generic until the first exchange completes.

Tables involved: *conversations*

3.2 Auto-Generate Conversation Title

Produces a short, descriptive title after the first exchange instead of leaving it generic.

Workflow

22. After the first user message and assistant reply are stored, both are sent to the LLM with a short summarization prompt.
23. The returned short phrase updates conversations.title.
24. This is a non-blocking enhancement — conversations remain fully usable with a default title if this step is skipped or delayed.

Tables involved: *conversations, messages*

3.3 List / Archive / Delete Conversations

Standard lifecycle management for a user's chat history.

Workflow

25. List: query conversations filtered by user_id, ordered by updated_at, paginated.
26. Archive: flip is_archived to true and exclude archived conversations from the default list view.
27. Delete: removing a conversations row cascades automatically to its messages, attachments, and conversation_documents links via foreign key constraints, so no manual cleanup step is required.

Tables involved: *conversations, messages, attachments, conversation_documents*

3.4 Full-Text Search Across Conversations

Lets a user find a past conversation by content rather than scrolling.

Workflow

28. A tsvector column is maintained on messages.content with a GIN index.
29. Search queries run Postgres full-text search against that index, scoped to the requesting user's conversations.
30. Results link back to the specific conversation and message for direct navigation.

Tables involved: *messages*

3.5 Export a Conversation

Downloads a conversation as a portable file.

Workflow

31. User requests an export of a specific conversation.
32. Server fetches all messages for that conversation_id in chronological order.

33. Messages are formatted into Markdown (or converted further to PDF) and returned as a downloadable file.

Tables involved: *conversations, messages*

4. Messaging

4.1 Send Message + Streamed LLM Response

The core chat loop.

Workflow

34. Client posts new message content to a conversation.
35. Server inserts a messages row with role = user.
36. Server fetches recent conversation history, truncated to fit the model's context window based on token count rather than row count.
37. If the conversation has scoped documents, RAG retrieval runs (see Section 6) and relevant chunks are injected into the prompt.
38. Server calls the LLM with the system prompt, history, and any retrieved context, streaming the response back to the client over SSE or WebSocket.
39. On completion, server inserts a messages row with role = assistant, recording model_name, token_count, and generation_time.
40. If retrieval was used, corresponding message_citations rows are inserted for every chunk actually included in the prompt.

Tables involved: *messages, conversations, message_citations*

4.2 Chat History Retrieval / Context Building

Assembles the conversation context sent to the LLM on each turn.

Workflow

41. Query messages for the conversation, ordered by created_at.
42. Apply a token-budget cutoff — older messages are dropped or summarized once the context window limit is approached.
43. Assembled history is combined with any retrieved RAG context and the system prompt before being sent to the model.

Tables involved: *messages, conversations*

4.3 Message Edit / Regenerate (Branching)

Allows editing an earlier message or regenerating a response without destroying history.

Workflow

44. User edits an earlier message, or requests regeneration of an assistant response.
45. A new messages row is inserted with parent_message_id pointing at the message being replaced, rather than overwriting the original row.
46. The client can reconstruct either branch by walking parent_message_id links, enabling a conversation-tree view even if the current UI only shows the latest branch.

Tables involved: *messages*

4.4 Feedback (Thumbs Up/Down + Text)

Captures qualitative signal on assistant responses.

Workflow

47. User rates an assistant message; server updates is_helpful (true, false, or left null if unrated).
48. Optional free-text feedback is stored in feedback_text.

Tables involved: *messages*

5. File & Document Handling

5.1 File Upload

Accepts PDF, DOCX, image, audio, and video files, either attached to a message or scoped to a conversation.

Workflow

49. Client uploads a file, either inline with a message or as a conversation-level document.
50. Server computes a SHA-256 content_hash of the file before any storage write.
51. If a document with that hash already exists, the upload is linked rather than re-stored (see 5.4).
52. Otherwise, the file is saved to disk or object storage and a new documents row is inserted with status = uploading.

Tables involved: *documents, attachments, user_documents, conversation_documents*

5.2 Upload Validation

Prevents oversized, malformed, or disguised files from entering the pipeline.

Workflow

53. Enforce a maximum file size before accepting the upload body.
54. Sniff actual file bytes to confirm the MIME type rather than trusting the client-supplied value.
55. Reject any file type outside the supported allowlist (PDF, DOCX, PNG, JPEG, MP4, MP3, and similar).

Tables involved: *documents, attachments*

5.3 Document Processing Pipeline

Turns an uploaded file into searchable, embedded text.

Workflow

56. A background worker picks up documents with status = processing.
57. Text is extracted with a format-appropriate library (PDF via pypdf, DOCX via python-docx, images via OCR if applicable).
58. Extracted text is split into chunks and inserted into document_chunks.
59. Each chunk is embedded using the system's pinned embedding model, and the resulting vector is inserted into chunk_embeddings.
60. documents.status is updated to ready, or failed with a logged reason if any step errors out.
61. The client polls or receives a push notification when status changes, so the UI can reflect progress.

Tables involved: *documents, document_chunks, chunk_embeddings, embedding_models*

5.4 Global Document Deduplication

Avoids re-storing, re-chunking, and re-embedding the same file across users or conversations.

Workflow

62. Before storing a new file, its content_hash is checked against the unique index on documents.content_hash.
63. If a match exists, the existing documents row is reused: a user_documents row links the uploader if not already linked, and a conversation_documents row scopes it into the current conversation.
64. No duplicate storage, extraction, or embedding work occurs for content that already exists in the system.

Tables involved: *documents, user_documents, conversation_documents*

5.6 Remove a Document from a Conversation

Unscopes a document from one conversation without necessarily deleting it system-wide.

Workflow

65. Server deletes the specific conversation_documents row linking that document to that conversation.
66. The underlying documents, document_chunks, and chunk_embeddings rows remain intact for reuse elsewhere.
67. A separate cleanup job periodically removes documents rows with no remaining user_documents or conversation_documents references, rather than deleting on every unlink.

Tables involved: *conversation_documents, documents*

6. Retrieval-Augmented Generation

6.1 Query Embedding + Similarity Search

Finds the most relevant chunks for a user's question.

Workflow

68. The user's query is embedded using the same pinned embedding model used at ingestion time.
69. A similarity search runs across `chunk_embeddings`, joined through `document_chunks` and scoped to the documents linked to the current conversation via `conversation_documents`.
70. Results are ordered by cosine distance using the HNSW index, and the top-k chunks are returned.

Tables involved: `chunk_embeddings`, `document_chunks`, `conversation_documents`, `embedding_models`

6.2 Re-ranking (Optional)

Improves precision on the initial similarity search results.

Workflow

71. The top-k chunks from vector search are passed to a cross-encoder re-ranking model.
72. Chunks are re-scored for relevance to the specific query and re-ordered.
73. A smaller final subset is selected for inclusion in the prompt, trading a small latency cost for better answer grounding.

Tables involved: `document_chunks` (*in-memory re-ranking, no schema change*)

6.3 Citation Tracking

Records which retrieved chunks actually informed a given answer.

Workflow

74. After the LLM generates a response using retrieved context, the chunks actually included in the prompt are identified.
75. A `message_citations` row is inserted per chunk, recording the `chunk_id`, `embedding_model_id`, `similarity_score`, and rank.
76. The composite foreign key into `chunk_embeddings` guarantees a citation can never reference a chunk/model pair that was never actually embedded.

Tables involved: `message_citations`, `chunk_embeddings`, `document_chunks`, `messages`

6.4 Prompt Injection Safeguards

Reduces the risk of retrieved document content overriding system instructions.

Workflow

77. Retrieved chunks are wrapped in the prompt template with an explicit label identifying them as untrusted reference material, not instructions.
78. The system prompt reinforces that instructions only come from the system role, not from retrieved content.
79. This is a mitigation, not a complete defense — retrieved content is still passed to the model and should be treated as an ongoing risk area.

Tables involved: `document_chunks` (*prompt-construction logic, no schema change*)

7. Platform & Operations

7.1 Rate Limiting / Abuse Prevention

Protects the system and its LLM costs from excessive or automated use.

Workflow

80. Each request to the message-send endpoint is checked against a per-user rate limit (token-bucket or sliding-window) backed by Redis.
81. Requests exceeding the limit are rejected before reaching the LLM call, avoiding unnecessary cost and load.

Tables involved: *Redis (no PostgreSQL table required)*

7.2 Usage & Token Quota Tracking

Tracks consumption for cost control and potential future billing.

Workflow

82. token_count is already recorded per message at generation time.
83. A background rollup aggregates token usage per user per day or month, stored in Redis counters or a dedicated summary table.
84. Quota checks can gate the message-send endpoint once a user's usage crosses a configured threshold.

Tables involved: *messages, usage_summary (new table, optional)*

7.3 Health & Readiness Endpoints

Standard operational endpoints for deployment and monitoring.

Workflow

85. /health confirms the API process itself is running.
86. /ready confirms the process can successfully reach PostgreSQL, Redis, and the LLM backend before being marked available to receive traffic.

Tables involved: *None (infrastructure-level check)*

7.4 Test Coverage

Guards the two areas where subtle bugs are most costly.

Workflow

87. Integration tests cover the full auth flow: sign up, login, refresh, logout, and revocation behavior.
88. Integration tests cover RAG retrieval: correct scoping to a conversation's documents, correct similarity ordering, and correct citation recording.

Tables involved: *All auth and RAG-related tables*